Machine Learning Intelligent Chip Design

指導教授: 陳坤志

HW4

NoC-based CNN architecture

(結報)

系級 中央大學電機四 B

學號 B132501

姓名 翁瑄妤

一、 模擬結果

主要在工作站上執行程式，使用 TA 提供的輸入資料（在 pattern 檔裡）進行測試。利用 SystemC 搭配自己實作的 4x4 Mesh-based NoC 系統，讓一整個 AlexNet 流程能在多個 PE 上運作，並將最終結果回傳給 Controller。

執行指令分為以下步驟:

(1) 打開所屬資料夾 (所需資料皆在 hw4 的資料夾裡)

```
cd hw4
```

(2) 執行 MakeFile (檔案由 TA 提供)

```
make
```

以下為執行結果:

經執行後有出現 ”Congratulations” 的字樣。上傳資料至工作站之測試結果也皆為正確。

```
Server_Account  mlchip074
Cat (35%)       0
Dog (35%)       0
Error_Message   No_Error
Submiss_Date    2025/05/31
Submiss_Time    01:14:35
Sim_Time (s)    2117.48


[Info] Your file will be submitted to:  TA folder
[Warning]  demo has been submitted.
[Warning]  It will overwrite your original file.
[Info] Now submit hw4_mlchip074.tar.gz file to system.
[Success] Copying Sucessfully.
=======================================================
                    Submit Report
=======================================================
Result          :   has been submitted.
Submission time :  2025/05/31 01:14:35
=======================================================
        ----------------------------
        --                        --       |\__||
        --  Congratulations !!    --      / O.O  |
        --                        --    /_____   |
        --  Submission Sucessful!!--   /^ ^ ^ \  |
        --                        --  |^ ^ ^ ^ |w|
        ----------------------------   \m___m__|_|
   Please remember to check your submission with ./02_check !!
   Please remember to check your submission with ./02_check !!
   Please remember to check your submission with ./02_check !!
=======================================================
```

模擬完成後，控制器成功輸出 top-100 的分類結果，並顯示各類別機率與名稱，代表 CNN 推論流程已順利完成。

二、 模組說明 (未使用 Virtual Channel)

1. Controller Design（controller.h、controller.cpp）

   功能：

   - 請求每一層所需的 input / weight / bias
   - 將 data 包裝成 flits，透過 NoC 傳至對應 PE
   - 最後接收結果，做 softmax 並輸出分類結果

   重點：

   - flit 結構修改：header 與 tail flit 只含 meta 資訊：
     - ◆ src：傳送 PE 編號

- ◆ dst：接收 PE 編號
- ◆ data_type：資料類型 (INPUT：輸入資料、WEIGHT：weight 資料、BIAS：bias 資料、OUTPUT：輸出至 Controller 的資料)

而真正的資料只存在 body flit，並使用 32-bit 來傳送 float。

- 每組資料打包格式為：
  - ◆ 第 1 個 flit：header（type=10）
  - ◆ 第 2～N 個 flit：body（type=00，存資料）
  - ◆ 最後 1 個 flit：tail（type=01）

```
void Controller::packet2flit(vector<float> data, int dst, int data_type){
    // packet to flits and give them to NoC (PE0, dest_id = 0)
    int src = 15;
    for (int i = 0; i < data.size() + 2; ++i) {
        sc_lv<34> flit;
        string type;
        if (i == 0) type = "10";  // header
        else if (i == data.size() + 1) type = "01";  // tail
        else type = "00";  // body

        flit.range(33, 32) = type.c_str();
        if (type == "10" || type == "01") {
            flit.range(31, 28) = sc_lv<4>(src);
            flit.range(27, 24) = sc_lv<4>(dst);
            flit.range(23, 22) = sc_lv<2>(data_type);  // use only 2 bits
            flit.range(21, 0)  = 0; // optional: padding
        }
        else {
            uint32_t bits;
            std::memcpy(&bits, &data[i - 1], sizeof(float));
            flit.range(31, 0) = sc_lv<32>(bits);  // full float body
        }

        req_tx.write(true);
        flit_tx.write(flit);
        do { wait(); } while (!ack_tx.read());
        req_tx.write(false);
    }
}
```

2. Core Design（core.h、core.cpp）

功能：

- 每個 PE 根據 core_id 對應執行 CNN 模型中的某一層。如下圖所示，其 core_id 值對應的工作與下圖 Layer 一致，例如 core 0 (Layer 0) 做 padding、core 1(Layer 1) 做 Convolutional Layer 1 + ReLU + Max Pooling 2、core 2(Layer 2) 做 Convolutional Layer 2 + ReLU + Max Pooling 2 …… 以此類推。

Layer	Type	Description	Output Dimension
0	Input Layer	Zero Padding 224 to 227	227x227x3
1	Convolutional Layer 1 + ReLU	64 kernels of 11x11, stride 4	55x55x64
1b	Max Pooling 1	3x3, stride 2	27x27x64
2	Convolutional Layer 2 + ReLU	192 kernels of 5x5, stride 1, padding 2	27x27x192
2b	Max Pooling 2	3x3, stride 2	13x13x192
3	Convolutional Layer 3 + ReLU	384 kernels of 3x3, stride 1, padding 1	13x13x384
4	Convolutional Layer 4 + ReLU	256 kernels of 3x3, stride 1, padding 1	13x13x256
5	Convolutional Layer 5 + ReLU	256 kernels of 3x3, stride 1, padding 1	13x13x256
5b	Max Pooling 5	3x3, stride 2	6x6x256
6	Fully Connected Layer 6 + ReLU	4096 neurons	4096
7	Fully Connected Layer 7 + ReLU	4096 neurons	4096
8	Fully Connected Layer 8	1000 neurons	1000

```
// 根據 PE ID 決定要執行哪一層
if (core_id == 0 && has_input) {
    // cout << "-----------------------------------" << endl;
    // cout << "core_id: " << core_id << " start!" << endl;
    // cout << "input_data.size(): " << input_data.size() << endl;
    result = zero_pad_flat(input_data, 3, 224, 224, 2, 2, 1, 1);
    // cout << "core_id: " << core_id << " done!" << endl;
    // cout << "result.size(): " << result.size() << endl;
    input_data.clear();
    weight.clear();
    bias.clear();
    send_req = true;
    has_input = false;
    has_weight = false;
    has_bias = false;
}
else if(has_input && has_weight && has_bias){
    // cout << "-----------------------------------" << endl;
    // cout << "core_id: " << core_id << " start!" << endl;
    // cout << "input_data.size(): " << input_data.size() << endl;
    if (core_id == 1) {
        // cout << "weight.size(): " << weight.size() << endl;
        // cout << "bias.size(): " << bias.size() << endl;
        vector<float> conv = conv_layer(input_data, weight, bias, 3, 227, 227, 11, 64, 4, 0);
        result = max_pool(conv, 64, 55, 55, 3, 2);
    }
    else if (core_id == 2) {
        vector<float> conv = conv_layer(input_data, weight, bias, 64, 27, 27, 5, 192, 1, 2);
        result = max_pool(conv, 192, 27, 27, 3, 2);
    }
    else if (core_id == 3) {
        result = conv_layer(input_data, weight, bias, 192, 13, 13, 3, 384, 1, 1);
    }
    else if (core_id == 4) {
        result = conv_layer(input_data, weight, bias, 384, 13, 13, 3, 256, 1, 1);
    }
    else if (core_id == 5) {
        vector<float> conv = conv_layer(input_data, weight, bias, 256, 13, 13, 3, 256, 1, 1);
        result = max_pool(conv, 256, 13, 13, 3, 2);
    }
    else if (core_id == 6) {
        result = fully_connect(input_data, weight, bias, 9216, 4096);
    }
    else if (core_id == 7) {
        result = fully_connect(input_data, weight, bias, 4096, 4096);
    }
    else if (core_id == 8) {
        result = fully_connect(input_data, weight, bias, 4096, 1000);
    }
```

重點：

- 接收時根據 header flit 解析目標 data_type，依照不同類型將資料存入 input_data, weight, bias 等 vector。
- 當某層需要的資料（input, weight, bias）齊全後，即開始執行該層運算，並產生 result。
- 結果打包成 flits 送往下一個 PE，若至 PE8 (core 8) 時則傳回 router 0，最後傳回 Controller。

```
// 發 flit 到下一層 PE (或 Controller)
if(send_req){
    do{ wait(); } while(req_rx.read());
    for (int i = 0; i < result.size() + 2; ++i) {
        sc_lv<34> flit;
        string type = (i == 0) ? "10" : (i == result.size() + 1) ? "01" : "00";

        flit.range(33, 32) = type.c_str();
        if(type == "10" || type == "01"){
            int num = 8;
            flit.range(31, 28) = sc_lv<4>(core_id);
            flit.range(27, 24) = (core_id < num) ? sc_lv<4>(core_id + 1) : sc_lv<4>(0); // next PE or Controller
            flit.range(23, 22) = (core_id < num) ? INPUT : OUTPUT;
        }
        else{
            uint32_t bits;
            std::memcpy(&bits, &result[i - 1], sizeof(float));
            flit.range(31, 0) = bits;
        }
        req_tx.write(true);
        flit_tx.write(flit);
        do { wait(); } while (!ack_tx.read());
        req_tx.write(false);
        // cout << "sending..." << endl;
        // wait();
    }
    send_req = false;
    // cout << "core send!" << endl;
    // cout << "-----------------------------------" << endl;
}
```

3. Router Design（router.h）

功能：

- 接收 flit，根據 header 的 dst 決定 routing 方向
- 使用 XY routing algorithm
- 維持五個方向 buffer，確保 flit 不會互相干擾 LOCAL port 連接函式:

重點：

- 加入 type 欄位判斷：若 type == OUTPUT 且 dst == 0，會將資料送往 Controller（透過 NORTH port）。
- routing 條件需考慮 header 鎖定與 tail 解鎖的流程，避免 pipeline 錯亂。

三、 Routing 演算法

仍使用 HW3 的 XY Routing：

- 若目標在右 → EAST
- 左 → WEST
- 到達同一行後再往 NORTH/SOUTH 移動
- 特殊情況：當要送回 Controller（PE16）時，會強制從 Router0 的 NORTH 方向送出

```
// calculate destination (route in x direction first, then y)
int Router::compute_dir(int dest, int type) {
    int x  = router_id % 4;
    int y = router_id / 4;
    int dx = dest % 4;
    int dy = dest / 4;

    if (router_id == 0 && type == OUTPUT) return NORTH;

    if (x < dx) return EAST;
    if (x > dx) return WEST;
    if (y < dy) return SOUTH;
    if (y > dy) return NORTH;
    return LOCAL;
}
```

四、 遇到的挑戰與解決方案

1. Flit 格式調整問題

   一開始沿用 HW3 的 flit 格式，但 float 資料無法完整傳輸，導致重組失敗。後來改為只在 body flit 傳資料，保留 32-bit 給 float，並將 header / tail flit 用於控制與 metadata，解決此問題。

2. Router 0 無法讀入最後結果

   觀察發現 Router 1 傳出 flit 至 Router 0 後無法進入其 output buffer。後來發現是因為 Router 0 尚未處理完上個 flit，造成 buffer 被塞滿。調整路由處理順序與 ack 條件後，問題解決。

五、 心得

這當初一看到 HW4 要把 CNN 架在 NoC 上跑，感覺很有挑戰性，因為除了要了解整個 CNN 的資料流程，還要設計好 Router、Controller、PE 等模組之間的通訊，整體比前幾

次作業複雜許多。一開始最難的地方是 flit 格式怎麼設計，特別是 float 資料該怎麼切、該放哪裡，弄錯的話整包資料都會錯，常常看到接收端拿到的數值怪怪的，花很多時間 debug。還有 Router 部分，一開始以為路由都寫好了就會動，但實際上 flit 要走完一整圈還要注意 buffer 有沒有被塞滿、ack 回來有沒有正常、資料有沒有對應到正確 PE，不然就會卡住不動。後來每加一層卷積或全連接層都得重新跑一次，看是不是資料有確實傳到下一層，超級花時間，但也因此更懂整個 pipeline 怎麼運作。最後看到 Controller 成功收到結果、印出分類結果的那一刻，真的有一種「終於通了」的爽感，覺得辛苦是值得的。這次作業讓我對 CNN 實作不只是停留在演算法的層面，而是實際知道怎麼在硬體模擬中去運作每一層，也讓我對 SystemC 的流程控制、資料封包傳輸有了更深的體會。